

Entregable 2: XML

Árbol General sobre Dataset GoodReads

Estructura de Datos — Semestre 1, 2026

Nicolás Renato Ricciardi Acuña
Enzo Gabriel Levancini Arriagada

25 de mayo de 2026

Índice

1. Introducción	2
2. Estructura del proyecto	2
3. Estructuras de datos	2
3.1. Book y SimilarBook	2
3.2. TreeNode	3
3.3. GeneralTree	3
4. Parseo de XML	3
5. Construcción del árbol	4
5.1. Problema: similitudes bidireccionales	4
5.2. Solución: DFS con conjunto de visitados	4
6. Función listar	5
7. Función borrar_ratings	6
7.1. Algoritmo post-order	6
8. Función precursores	6
8.1. Manejo de años no registrados	6
9. Estadísticas del árbol	7
10. Compilación y ejecución	7
10.1. Requisitos	7

10.2. Pasos	7
11.Decisiones de diseño	8

1. Introducción

Este proyecto implementa un **árbol general** en C++17 a partir de 10.000 archivos XML del dataset GoodReads. Cada archivo describe un libro con sus metadatos (título, ISBN, rating, año de publicación, etc.) y una lista de libros similares que define la estructura jerárquica del árbol.

El sistema soporta tres operaciones principales sobre el árbol:

1. `listar` — recorrido preorder que imprime los IDs de todos los libros.
2. `borrar_ratings(r)` — elimina todos los libros con rating promedio $\leq r$.
3. `precursores(id)` — lista los libros cuyo año de publicación es menor al de *todos* sus libros similares.

2. Estructura del proyecto

```
Tarea2/
  CMakeLists.txt      <- configuración de compilación (CMake + C++17)
  README.md
  data/               <- 10.000 archivos .xml del dataset GoodReads
  libs/
    pugixml/          <- librería PugiXML para parsear XML
  src/
    Book.h            <- structs Book y SimilarBook
    TreeNode.h        <- nodo del árbol general
    GeneralTree.h/.cpp <- árbol general con las tres funciones requeridas
    XMLParser.h/.cpp  <- carga y parseo de archivos XML
    main.cpp          <- punto de entrada
```

3. Estructuras de datos

3.1. Book y SimilarBook

SimilarBook almacena la información mínima de un libro referenciado en la sección `<similar_books>` del XML. Book contiene todos los campos relevantes de un libro, incluyendo su lista de similares.

```
1 struct SimilarBook {
2     std::string id;
3     std::string title;
4     std::string isbn;
5     int         publicationYear;
6 };
7
8 struct Book {
```

```

9      std::string      id;
10     std::string      title;
11     std::string      isbn;
12     int               publicationYear;
13     std::string      language;
14     std::string      description;
15     double            averageRating;
16     int               numPages;
17     std::vector<SimilarBook> similarBooks;
18 };

```

3.2. TreeNode

Cada nodo del árbol almacena un `Book` y un vector de punteros a sus hijos. Se eligió `std::vector` sobre la representación hijo-izquierdo/hermano-derecho para simplificar los recorridos iterativos.

```

1 struct TreeNode {
2     Book          book;
3     std::vector<TreeNode*> children;
4
5     explicit TreeNode(const Book& b) : book(b) {}
6 };

```

3.3. GeneralTree

La clase principal del árbol mantiene:

- `root_` — raíz virtual con `Book` vacío, punto de entrada único.
- `nodeMap_` — tabla hash `unordered_map<string, TreeNode*>` para buscar cualquier nodo por ID en tiempo $O(1)$ durante la construcción.

4. Parseo de XML

Se utiliza la librería **PugiXML** para leer los archivos XML. La estructura real de cada archivo en el dataset es:

```

<GoodreadsResponse>
  <book>
    <id>...</id>
    <title>...</title>
    <publication_year>...</publication_year>
    <average_rating>...</average_rating>
    <similar_books>
      <book>

```

```

        <id>...</id>
        <publication_year>...</publication_year>
    </book>
</similar_books>
</book>
</GoodreadsResponse>

```

XMLParser::parseFile() extrae cada campo usando child_value() de PugiXML y convierte los valores numéricos con lambdas defensivas que retornan 0 ante campos vacíos, evitando excepciones:

```

1 auto toInt = [](const char* s) -> int {
2     try { return (s && *s) ? std::stoi(s) : 0; }
3     catch (...) { return 0; }
4 };

```

XMLParser::loadDirectory() itera recursivamente la carpeta data/ con std::filesystem y muestra una barra de progreso durante la carga.

5. Construcción del árbol

5.1. Problema: similitudes bidireccionales

En el dataset GoodReads, las relaciones de similitud son **simétricas**: si el libro A lista a B como similar, B también lista a A. Esto convierte el conjunto de relaciones en un **grafo con ciclos**, que no puede representarse directamente como árbol.

5.2. Solución: DFS con conjunto de visitados

Para convertir el grafo en un árbol sin ciclos se aplica un **DFS iterativo** sobre todos los nodos. Cada nodo se incorpora al árbol exactamente una vez (la primera vez que es alcanzado), garantizando que no se formen ciclos:

```

1 void GeneralTree::build() {
2     std::unordered_set<std::string> visited;
3
4     for (auto& [startId, startNode] : nodeMap_) {
5         if (visited.count(startId)) continue;
6         visited.insert(startId);
7         std::stack<TreeNode*> pila;
8         pila.push(startNode);
9
10        while (!pila.empty()) {
11            TreeNode* curr = pila.top(); pila.pop();
12            for (const SimilarBook& sb : curr->book.similarBooks)
13                {
14                    auto it = nodeMap_.find(sb.id);

```

```

14         if (it != nodeMap_.end() && !visited.count(sb.id)
15             ) {
16             visited.insert(sb.id);
17             curr->children.push_back(it->second);
18             pila.push(it->second);
19         }
20     }
21 }
22 // Nodos sin padre -> hijos de la raíz virtual
23 std::unordered_set<std::string> claimed;
24 for (auto& [id, node] : nodeMap_)
25     for (const TreeNode* hijo : node->children)
26         claimed.insert(hijo->book.id);
27
28 for (auto& [id, node] : nodeMap_)
29     if (!claimed.count(id))
30         root_->children.push_back(node);
31 }

```

Los nodos que ningún otro nodo reclama como hijo se colocan directamente bajo la raíz virtual, formando un **bosque de expansión** con un único punto de entrada.

6. Función listar

Recorre el árbol en **preorder** (nodo actual antes que sus hijos) e imprime el ID de cada libro. El recorrido es **iterativo** usando una pila explícita para evitar desbordamiento de pila con árboles profundos.

Los hijos se insertan en la pila en **orden inverso** para que al hacer *pop* se procesen de izquierda a derecha, manteniendo el orden preorder correcto.

```

1 void GeneralTree::listar(std::ostream& os) const {
2     std::stack<const TreeNode*> pila;
3     for (auto it = root_->children.rbegin();
4         it != root_->children.rend(); ++it)
5         pila.push(*it);
6
7     while (!pila.empty()) {
8         const TreeNode* node = pila.top(); pila.pop();
9         os << node->book.id << "\n";
10        for (auto it = node->children.rbegin();
11            it != node->children.rend(); ++it)
12            pila.push(*it);
13    }
14 }

```

Complejidad: $O(n)$, donde n es el número de nodos.

7. Función borrar_ratings

Elimina del árbol todos los nodos cuyo `averageRating` $\leq r$. Al eliminar un nodo se elimina también todo su subárbol (sus descendientes pierden su único camino hacia la raíz).

7.1. Algoritmo post-order

Se usa un recorrido **post-order recursivo**: se evalúan los hijos *antes* que el padre. Esto permite liberar la memoria de los subárboles de abajo hacia arriba sin perder referencias.

```
1 bool GeneralTree::borrarRatingsHelper(TreeNode* node, double r) {
2     std::vector<TreeNode*> sobrevivientes;
3     for (TreeNode* hijo : node->children) {
4         if (!borrarRatingsHelper(hijo, r))
5             sobrevivientes.push_back(hijo);
6         else
7             deleteSubtree(hijo);
8     }
9     node->children = sobrevivientes;
10    return node->book.averageRating <= r;
11 }
```

La función auxiliar retorna true si el nodo debe ser eliminado. El llamador se encarga de liberarlo con `deleteSubtree()`.

Complejidad: $O(n)$.

8. Función precursores

Lista los IDs de todos los libros que cumplen la siguiente condición:

$$\forall s \in \text{similarBooks}(v) : s.\text{publicationYear} > v.\text{publicationYear}$$

Es decir, el libro fue publicado *antes* que todos sus libros similares, y tiene al menos un libro similar.

8.1. Manejo de años no registrados

Si un libro similar tiene `publicationYear == 0` (campo vacío en el XML), se trata como *anterior* al libro evaluado, excluyéndolo del resultado. Esto evita falsos positivos por datos faltantes.

```
1 for (const SimilarBook& sb : sibs) {
2     // a o 0 = no registrado; se trata como anterior
3     if (sb.publicationYear == 0 ||
```

```

4         sb.publicationYear <= node->book.publicationYear) {
5             todos_posteriores = false;
6             break;
7         }
8     }

```

El recorrido es **iterativo** sobre todo el árbol, usando la misma técnica de pila explícita que `listar`.

Complejidad: $O(n \cdot m)$, donde m es el promedio de libros similares por nodo.

9. Estadísticas del árbol

La función `stats()` calcula en un único recorrido $O(n)$:

Métrica	Descripción
Nodos totales	Cantidad de libros en el árbol
Altura máxima	Profundidad del nodo más lejano de la raíz virtual
Promedio de hijos	Total de hijos / total de nodos
Libro más conectado	ID del nodo con mayor cantidad de hijos directos

Cuadro 1: Métricas calculadas por `stats()`

10. Compilación y ejecución

10.1. Requisitos

- CMake ≥ 3.16
- Compilador con soporte C++17 (GCC 8+, Clang 7+, MSVC 2017+)

10.2. Pasos

```

mkdir build
cd build
cmake ..
cmake --build .
./tarea2

```

Los resultados se guardan automáticamente en tres archivos dentro de `build/`:

- `listar.txt` — IDs en preorder
- `listar_post_borrado.txt` — IDs tras eliminar libros con $\text{rating} \leq 3,5$
- `precursores.txt` — IDs de libros precursores

11. Decisiones de diseño

Raíz virtual

Se usa un nodo raíz con `Book` vacío en lugar de elegir un libro arbitrario como raíz real. Esto permite que el árbol sea un *bosque* sin forzar una jerarquía artificial.

Recorridos iterativos

`listar` y `precursores` usan pila explícita en lugar de recursión para evitar desbordamiento de pila con 10.000 nodos.

DFS para romper ciclos

La alternativa más simple habría sido ignorar los duplicados en `children`, pero el DFS con `visited` garantiza un árbol válido sin nodos repetidos.

Tabla hash para construcción

El `unordered_map` permite buscar cualquier libro por ID en $O(1)$ durante `build()`, manteniendo la construcción total en $O(n \cdot m)$.